

■ 과제 1번.cpp (5개 실수의 평균, 최댓값, 최소값 구하기)

• 어려운 점 (Problem)

사용자가 입력할 실수의 범위(양수, 음수, 0 등)를 개발 단계에서 예측할 수 없기 때문에, 최댓값(max)과 최소값(min)의 초기값을 단순히 0이나 임의의 상수로 설정하면 안 됩니다. 만약 초기 최댓값을 0으로 설정했는데 사용자가 모두 음수만 입력한다면, 프로그램은 실제 최댓값인 음수가 아닌 초기값 0을 최댓값으로 출력하는 치명적인 논리 오류를 범하게 됩니다.

• 해결 방법 (Solution)

사용자로부터 입력을 모두 받아 배열에 저장한 후, 배열의 첫 번째 값인 numbers[0]을 최댓값과 최소값의 절대적인 초기 비교 기준으로 설정했습니다. 이후 반복문을 2번째 요소(Index 1)부터 수행하며 값을 비교하고 갱신하도록 설계하여, 입력값의 범위와 상관없이 항상 정확한 통계치가 도출되도록 해결했습니다.

■ 과제 2.cpp (피보나치 수열 항 구하기)

• 어려운 점 (Problem)

피보나치 수열은 항의 번호가 커질수록 그 값이 기하급수적으로 증가합니다. C언어에서 흔히 사용하는 32비트 정수형(int) 변수를 사용하면 약 47번째 항만 도달하더라도 정수 오버플로우(Integer Overflow)가 발생하여 값이 음수로 뒤집히거나 비정상적인 데이터가 출력되는 한계가 있습니다.

• 해결 방법 (Solution)

수열의 계산값을 누적해 나가는 핵심 변수(n1, n2, next)를 모두 64비트 표현 범위를 가지는 long long 자료형으로 선언하였습니다. 이를 통해 연산 안정 범위를 극대화하였고, 사용자가 0 이하의 숫자를 입력할 때를 대비하여 최상단에 예외 처리 코드를 배치함으로써 런타임 에러를 사전에 차단했습니다.

■ 과제 3.cpp (윤년 판별 프로그램)

• 어려운 점 (Problem)

윤년 판별법은 "4의 배수여야 함", "100의 배수가 아니어야 함", "그럼에도 400의 배수이면 윤년임"이라는 상호 배타적이면서 종속적인 여러 조건이 얹혀 있습니다. 이를 중첩된 조건문(nested if-else)으로만 작성할 경우, 코드가 복잡해지고 조건 판별의 누락이 발생하기 쉬워집니다.

• 해결 방법 (Solution)

C언어의 논리 연산자인 && (AND)와 || (OR)의 연산 우선순위를 정확히 파악하고 이를 적용하여, 단 하나의 조건식 ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))으로 결합했습니다. 이로 인해 불필요한 분기 흐름을 걷어내고 코드 가독성을 극대화했습니다.

■ 과제 4.cpp (사칙연산 및 거듭제곱 계산기)

• 어려운 점 (Problem)

제공된 소스 코드 내에서 안내 문구에 따라 printf와 scanf를 이용해 순차적으로 값을 입력받도록 구성해 두었으나, 바로 밑에 if (scanf("%lf %c %lf", ...)) 형태로 일괄 입력을 한 번 더 받는 중복 코드가 존재했습니다. 이로 인해 입력 버퍼에 잔여 문자가 남아 프로그램이 멈추거나 두 번 입력해야만 진행되는 버퍼 충돌이 발생했습니다. 또한, 나눗셈 연산 시 분모가 0이 되는 예외 상황에 대한 안전장치가 필요했습니다.

• 해결 방법 (Solution)

입력 버퍼의 비정상적인 작동을 막기 위해 입력 방식을 하나로 통일할 것을 제안합니다. 코드 내부에서는 나누는 수(분모)가 0인 상황을 if (num2 == 0) 조건식으로 사전에 필터링하고, 에러 메시지를 출력한 뒤 is_valid = 0 플래그 처리를 수행하여 안전하게 연산을 종료하도록 안전망을 구축했습니다.

■ 과제 5.cpp (모래시계 반대 모양 별 찍기)

• 어려운 점 (Problem)

가운데가 비어 있고 좌우가 대칭인 모래시계 반대 모양(나비 모양)을 만들기 위해서는 한 행 내에서 "왼쪽 별의 개수 - 가운데 공백의 개수 - 오른쪽 별의 개수"의 상관관계를 통제해야 합니다. 줄이 내려감에 따라 공백은 줄어듦과 별은 늘어나는 구조적 규칙과, 중간 행을 기점으로 다시 형태가 반전되는 대칭을 수식으로 처리하는 것이 까다로웠습니다.

• 해결 방법 (Solution)

행의 위치에 따른 공백과 별의 개수 규칙을 명확히 수식화했습니다. 출력 루프를 상단부(1행~N행)와 하단부(N-1행~1행 역순)로 과감히 이중 분할하였습니다. 한 행에 출력되는 좌우 별은 각각 i 개이며, 가운데 공백은 $2 * (N - i)$ 개라는 산술 공식을 도출하여 적용함으로써 코드 구조를 비약적으로 단순화했습니다.

■ 과제 6.cpp (속이 빈 삼각형 별 찍기)

• 어려운 점 (Problem)

삼각형의 가장자리 윤곽선만 별로 채우고 내부를 빈칸으로 구현해야 하는 과제입니다. 첫 번째 행(별 1개), 마지막 행(가득 찬 바닥면, 별 $2n-1$ 개), 그리고 그 사이의 중간 행들(시작 별 + 빈 공간 + 끝 별)이 모두 서로 다른 출력 메커니즘을 요구하므로 단일 반복문으로 제어하기가 불가능에 가까웠습니다.

- **해결 방법 (Solution)**

출력하는 행의 위치(i)에 따라 분기 처리를 3단계로 명확히 나누었습니다.

1. $i == 1$ (첫 행)일 때는 별 1개만 출력하고 개행합니다.
2. $i == n$ (마지막 행)일 때는 바닥면 구성을 위해 $2 * n - 1$ 개의 별을 반복문으로 출력합니다.
3. 그 외의 사이줄(중간 행)은 첫 별을 찍은 뒤, $2 * i - 3$ 개만큼의 공백을 출력하고 마지막 별을 찍도록 규칙적인 수식을 설계하여 속이 빈 삼각형을 완벽히 묘사했습니다.

■ 과제 7.cpp (경우의 수 계산기: nPr , nPr , nCr , nHr)

- **어려운 점 (Problem)**

순열(nPr), 조합(nCr), 중복조합(nHr) 등 다양한 경우의 수를 구하는 과정에서 팩토리얼 연산이 다수 포함됩니다. 이 과정에서 중간 계산 과정의 숫자가 과도하게 커져 값이 깨지는 정수 초과 에러가 빈번히 발생했습니다. 특히 조합을 계산할 때 나눗셈 연산을 먼저 처리해버리면 소수점이 잘려 나가 최종 결과 값이 완전히 왜곡되는 현상이 동반되었습니다.

- **해결 방법 (Solution)**

모든 계산용 누적 변수를 long long 자료형으로 일괄 선언하여 연산의 상한 임계치를 높였습니다. 조합(nCr)을 계산할 때 분자 연산인 순열(nPr)의 최종 결과값을 구한 뒤, 마지막에 분모인 $r!$ 값으로 나누는 순서적 최적화를 적용하여 소수점 손실 문제를 해결했습니다. 아울러 $n < r$ 인 경우 등 수학적으로 계산이 불가능한 예외 입력값에 대해 사전에 경고 메시지를 띄우고 프로그램을 종료하도록 방어 코드를 적용했습니다.